

Normalisation orthographique du moyen français

Chapitre 4 — règles, lexiques et fine-tuning

Pourquoi commencer par les règles ?

- * Empirique : 60 à 75 % des écarts HTR → référence sont des alternances graphiques systématiques, résolubles sans modèle
- * Épistémique : les règles sont traçables, justifiables et réversibles — exigence des humanités numériques
- * Instrumentale : les règles servent de ligne de base (baseline) pour juger si le fine-tuning apporte un vrai gain

Un modèle neuronal qui réapprend ce que les règles savent déjà gaspille sa capacité sur du bruit structuré.

Les quatre étapes du TP

1. Règles graphiques

Pipeline de fonctions pures : u/v, i/j, terminaisons

2. Abréviations

Résolution des tildes et formes suscrites résiduelles

3. Arbitrage MLM

CamemBERT tranche les positions de faible confiance

4. Fine-tuning mT5 LoRA

Le modèle neuronal traite ce qui reste

PARTIE 1

Éléments de linguistique du moyen français

Le moyen français, c'est quoi ?

- * Français écrit entre environ 1330 et 1500
- * Langue des chroniques (Froissart, Commines), des chartes, comptes et registres administratifs
- * C'est la matière première de vos corpus HTR

Deux propriétés à retenir absolument pour la suite : l'orthographe n'est pas standardisée, et la morphologie reste flexionnelle.

Une orthographe non standardisée

- * Pas d'Académie française au XIVe siècle
- * Un même mot peut s'écrire de cinq façons différentes selon le scribe, sa région, sa fatigue
- * Exemple : « roi » s'écrit roi, roy, roÿ, roys, rei, rois...

Ce ne sont pas des fautes : c'est la norme de l'époque. La normalisation ne « corrige » pas un texte fautif, elle harmonise des variantes légitimes.

Une morphologie encore flexionnelle

- * Trace du système de déclinaison latin
- * Cas sujet (li roys) vs cas régime (le roi) pour les noms masculins
- * La distinction li / le est une information morphosyntaxique, pas une erreur

Attention : toutes les alternances ne doivent pas être « corrigées » automatiquement — certaines portent du sens grammatical.

Signifiant, signifié, lemme

Le vocabulaire de Saussure appliqué à la normalisation

- * Signifiant : la forme graphique ou sonore d'un mot
- * Signifié : le concept porté par le mot
- * Un même signifié peut avoir des dizaines de signifiants graphiques différents en moyen français

La normalisation ramène ces variantes de signifiant vers une forme canonique : le lemme.

Le lemme et le morphème

Lemme

La forme de citation : infinitif pour un verbe, cas régime singulier pour un nom médiéval (roi, pas li roys)

Morphème

La plus petite unité de sens : dans signait → racine sign- + désinence -ait (imparfait)

Pour la normalisation, ce qui compte est la racine graphique (identifie le lemme) et les affixes flexionnels (encodent l'information grammaticale).

Quatre niveaux de variation graphique

Hiérarchisés par régularité décroissante

Niveau 1 — Alternances phonographiques

u/v, i/j, c/k/qu — résolues par des règles de position

Niveau 2 — Variantes morphographiques

-oit/-ait, -our/-oir — résolues par table de correspondances

Niveau 3 — Variantes lexicales régionales

chastel (picard) vs château — nécessitent un lexique de référence

Niveau 4 — Abréviations conventionnelles

Traitées séparément à l'étape 2

Type, forme, lemme : vocabulaire opérationnel

Ces trois notions apparaissent directement dans le code du TP

Type

Une forme graphique distincte. Roys, roy, roi, rois sont quatre types différents.

Forme (token)

Une occurrence d'un type dans le texte. Si roys apparaîtrait 17 fois, c'est 1 type mais 17 tokens.

Lemme

La forme canonique qui regroupe tous ces types. Roi est le lemme de roys, roy, roi, rois, rei...

La normalisation = mapper des types médiévaux vers leurs lemmes canoniques (ou leurs équivalents en français moderne).

Alternance u/v

u et v sont deux graphies de la même lettre

- * Le son [v] en position médiale s'écrit souvent u : auoir (avoir), sauoir (savoir)
- * Règle positionnelle : u initial devant voyelle, ou u entre consonne et voyelle → v
- * u en position finale ou après voyelle reste u

auoir → avoir · sauoir → savoir · uoir → voir

Coder l'alternance u/v

Une fonction pure, testable indépendamment

```
def normalize_uv(word: str) -> str:
    # u initial devant voyelle : auoir -> avoir
    word = re.sub(r'^u(?=[aeiou...])', 'v', word)
    # u médial entre consonnes/voyelle : sauoir -> savoir
    word = re.sub(r'(?<=[bcd fg...])u(?=[aeiou...])', 'v', word)
    return word
```

- * Chaque règle est une fonction pure : entrée = chaîne, sortie = chaîne normalisée
- * Testable de façon isolée avec des assertions (auoir → avoir, sauoir → savoir)

Alternance i/j

Symétrique à u/v

- * i en position initiale devant voyelle représente le son [j]
- * iuger → juger · iour → jour · iustice → justice
- * Cas inverse plus rare : j médial devant consonne → i (aijde → aide)

Même logique positionnelle que u/v — une règle de voisinage, pas une exception au cas par cas.

Terminaisons désinentielles variables

Résolues par table de correspondances, pas par règle de position

- * Imparfait : -oit / -ait selon le scribe
- * Conditionnel : -roit / -rait
- * Substantifs : -eur / -our / -oir
- * Alternance c/k/qu : le son [k] s'écrit c devant a/o/u, qu devant e/i, parfois k (influence nordique)

PARTIE 2

Étape 1 — Module de règles graphiques

Architecture du pipeline de règles

Cinq étapes séquentielles, dans un ordre précis

- 1 Normalisation Unicode (NFC + caractères médiévaux spéciaux)
- 2 Alternances graphiques systématiques (u/v, i/j, c/k/qu)
- 3 Terminaisons variables (table de correspondances)
- 4 Lookup DMF / LGeRM (vérification de la forme normalisée)
- 5 Mesure du CER avant/après

Chaque règle est une fonction pure ; certaines dépendent du résultat des précédentes — l'ordre est une décision linguistique, pas un détail technique.

Étape 1 : la normalisation Unicode

Un problème souvent invisible

- * Les outils HTR n'encodent pas toujours les caractères médiévaux de façon cohérente
- * NFC (Normalisation Canonique) unifie les représentations équivalentes : é peut être un seul codepoint ou e + accent combinant
- * Des caractères médiévaux spéciaux (p barré, j sans point, i sans point...) doivent être translittérés vers leurs équivalents

Table de translittération médiévale

Un dictionnaire simple : caractère spécial → équivalent standard

```
MEDIEVAL_UNICODE = {  
    'p̄': 'p', # p barré (per/par/pro)  
    'j': 'j', # j sans point  
    'i': 'i', # i sans point  
    '\u0304': '', # macron combinant (nasale, traité étape 2)  
}
```

unicodedata.normalize('NFC', text) s'applique avant la translittération, pour garantir une base homogène.

La table de correspondances graphiques

Le cœur du module de règles

- * Liste de paires (motif → remplacement) appliquées dans l'ordre
- * Construite à partir du DMF et de la convention t9n
- * Exemples : -oit → -ait (imparfait), -our → -oir, -aus → -aux

```
GRAPHEME_RULES = [  
    (r'oit\b', 'ait'), # imparfait : -oit -> -ait  
    (r'our\b', 'oir'), # vouloir, pouvoir, savoir  
    (r'aus\b', 'aux'), # vieux, eaux  
]
```

L'ordre des règles est une décision linguistique

- * Les règles plus longues / spécifiques doivent précéder les règles plus courtes / générales
- * Appliquer -oit → -ait avant ou après une autre règle peut changer le résultat final
- * Dans ce module, l'ordre n'est pas un détail d'implémentation

Toute décision sur l'ordre des règles doit être documentée et justifiée dans CONVENTIONS_NLP.md.

Mesurer le CER : Character Error Rate

La distance d'édition normalisée par la longueur de la référence

$$CER = (S + D + I) / N$$

- * S = substitutions, D = suppressions, I = insertions de caractères
- * N = longueur de la référence
- * 0 = parfait, 1 = entièrement faux

La réduction de CER obtenue par les règles seules est votre baseline pour juger le fine-tuning.

Interpréter la réduction de CER

15 – 30 %

Réduction typique sur un corpus de chartes normandes — règles efficaces

< 10 %

Règles trop conservatrices, ou CER initial déjà bas

> 40 %

Vérifier que la référence est bien du français moderne et non du moyen français
« propre »

PARTIE 3

Le Dictionnaire du Moyen Français (DMF) et LGeRM

Qu'est-ce que le DMF ?

- * Dictionnaire de référence scientifique élaboré par l'ATILF (CNRS – Université de Lorraine)
- * Couvre le français des XIVe et XVe siècles (1330–1500)
- * Plus de 65 000 entrées, 470 000 exemples contextuels, définitions en français moderne
- * Équivalent pour le moyen français du Trésor de la Langue Française

Adresse pérenne : www.atilf.fr/dmf

Le double rôle du DMF dans votre pipeline

Forme lemmatisée canonique

Donne le lemme de référence pour chaque mot médiéval

Variantes graphiques attestées

Documente les formes connues pour chaque lemme

Avant d'écrire une règle, vérifiez sa cohérence avec le DMF : si chastel → château, le DMF doit confirmer que château est bien le lemme et chastel une variante attestée.

LGeRM : le lemmatiseur intégré au DMF

Lemmes, Graphies et Règles Morphologiques

- * Développé par Gilles Souvay (ATILF), intégré à l'interface du DMF
- * Combine une base de formes connues lemmatisées et des règles graphémiques/morphologiques médiévales
- * Conçu pour le moyen français (1330-1500), puis adapté aux XVIe-XVIIe siècles

Exemple : taper « roys » dans le DMF déclenche LGeRM, qui propose le lemme roi avec variantes et définitions.

Accès programmatique au DMF

Pas d'API REST officielle

- * Le DMF n'expose pas d'API documentée
- * Accès par scraping HTML de l'interface web
- * Respecter les conditions d'utilisation de l'ATILF et limiter le débit des requêtes (politesse réseau)

```
def query_dmf(forme: str, delay: float = 1.0) -> dict:
    time.sleep(delay) # politesse vis-à-vis du serveur ATILF
    resp = requests.get(DMF_SEARCH_URL.format(forme=forme.lower()))
    # ... extraction du lemme, définition, variantes via BeautifulSoup
```

Avertissement pratique sur le scraping

- * L'interface du DMF est une application web CGI héritée
- * La structure HTML peut varier entre versions (DMF2012, DMF2020, DMF2023)
- * Si les sélecteurs CSS ne retournent rien, inspecter la page manuellement avec `soup.prettify()` et adapter

Constituer un cache local du DMF

Un cache JSON est aussi un artefact de traçabilité

- * Évite de solliciter le serveur ATILF à chaque exécution du pipeline
- * Documente exactement quelles requêtes ont produit quels résultats, à quelle date
- * Pré-remplissage : ne traiter que les ~200 types les plus fréquents — assez pour couvrir 80 % des tokens

```
# data/dmf_cache.json — { "roys": {"lemme": "roi", ...}, ... }  
def get_dmf_lemme(forme, cache):  
    if forme in cache: return cache[forme].get("lemme")  
    ...
```


Combiner règles graphémiques et lookup DMF

Le DMF valide et complète ce que les règles ne couvrent pas

- 1 Appliquer les règles graphémiques (u/v, i/j, terminaisons)
- 2 Chercher la forme originale dans le cache DMF
- 3 Si trouvé → retourner le lemme DMF
- 4 Si non trouvé → fallback sur le résultat des règles (recommandé en production)

Le fallback est essentiel : le DMF ne couvre pas tous les hapax ni les noms propres.

PARTIE 4

Étape 2 — Résolution des abréviations résiduelles

Quatre familles d'abréviations médiévales

Conventions calligraphiques héritées de la pratique latine

Suspension

La fin du mot est coupée. S. pour saint, m. pour monsieur

Contraction

Lettres médianes supprimées, reliées par un trait. norm[~]die pour normandie

Lettre suscite

Lettre écrite en exposant. m^r pour monseigneur, s^r pour seigneur

Signes suprascripts spéciaux

Tilde de nasalité, p barré (**p**) pour par/per/pro

Le tilde de nasalité

La consonne nasale supprimée se reconstruit par contexte

- * Devant b ou p : la nasale supprimée est m → co~pte devient compte
- * Devant toute autre consonne : la nasale est n → norm~die devient normandie
- * C'est une règle phonologique déterministe dans la grande majorité des cas

co~bat → combat · te~ps → temps · fra~ce → france

Coder la résolution du tilde

```
def resolve_nasal_tilde(word: str) -> str:
    # ~ suivi de b ou p -> m
    word = re.sub(r'~(?=[bp])', 'm', word)
    # ~ suivi d'une autre consonne -> n
    word = re.sub(r'~(?=[cdfghjklrstvwxyz])', 'n', word)
    return word
```

Un troisième cas — le tilde en fin de mot ($q^{\sim}$) — reste ambigu et nécessite une table contextuelle.

La table de résolution contextuelle

Quand la phonologie seule ne suffit pas

- * Construite à partir du DMF et des conventions du corpus
- * Organisée par domaine : pronoms, formules latines, titres, géographie, monnaies
- * Exemples : q~ → que, m^r → monseigneur, l~t → livres tournois

```
ABBREV_TABLE = {  
    "q~": "que", "m^r": "monseigneur",  
    "norm~die": "normandie", "l~t": "livres tournois",  
}
```

Stratégie de résolution combinée

Phonologie d'abord, table ensuite

- 1 Appliquer la résolution phonologique (tilde)
- 2 Chercher la forme originale dans la table
- 3 Sinon, chercher la forme phonologique dans la table
- 4 Sinon, retourner la forme phonologique si elle diffère, ou signaler non résolu

La fonction retourne aussi un booléen `was_resolved` : utile pour annoter les cas ambigus dans `CONVENTIONS_NLP.md`.

Documenter les choix

CONVENTIONS_NLP.md : un livrable scientifique, pas un fichier optionnel

- * Toute décision non entièrement déterminée par le DMF doit y figurer, avec sa justification
- * Format attendu : forme brute, expansion choisie, alternatives écartées, justification
- * Exemple : $p \sim \rightarrow \text{par}$ (alternatives écartées : per, pro, pré), justifié par le contexte devant un substantif

Versionné en sémantique : v1.0.0 (version initiale), v1.0.1 (ajout de règle), v1.1.0 (modification d'une règle existante).

PARTIE 5

Étape 3 — Arbitrage par CamemBERT (MLM)

Le problème : positions de faible confiance

Le champ « candidates » du data contract HTR

- * Pour certaines positions, le HTR hésite entre deux caractères et liste les alternatives avec leurs scores
- * Exemple : ~ transcrit avec 60% de confiance, mais a était l'alternative à 40%
- * Après résolution : norm~die → normandie. Mais avec a → normadie, une forme inexistante dans le DMF

Idée : utiliser CamemBERT en mode masked language model pour arbitrer entre les deux hypothèses.

La pseudo-log-vraisemblance (PLL)

Une approximation tractable de la probabilité d'un texte

$$PLL(x) = \sum_i \log P(x_i \mid x_{-i}; \vartheta)$$

- * On masque chaque token un par un, et on somme les log-probabilités du token original
- * Plus la PLL est élevée, plus le texte est « probable » selon le modèle

Référence : Salazar et al. (2020), « Masked Language Model Scoring », ACL 2020.

Algorithme d'arbitrage en trois étapes

- 1 Identifier les positions candidates (confidence < seuil)
- 2 Construire deux versions de la ligne, une par alternative
- 3 Scorer les deux versions avec CamemBERT (PLL) et choisir la meilleure

Seuil de confiance recommandé : 0.70 (cohérent avec le Chapitre 2).

Construire les deux versions à arbitrer

```
version_a = trans[:pos] + alt_a + trans[pos+len(alt_a):]  
version_b = trans[:pos] + alt_b + trans[pos+len(alt_b):]  
  
score_a = score_text_mlm(resolve_nasal_tilde(version_a))  
score_b = score_text_mlm(resolve_nasal_tilde(version_b))  
chosen = version_a if score_a >= score_b else version_b
```

alt_a est la transcription HTR la plus probable, alt_b l'alternative — on résout phonologiquement avant de scorer.

Trois stratégies d'arbitrage à comparer

Une ablation directement liée au Chapitre 3

A — Confiance seule

On fait confiance au HTR au-dessus du seuil, et on applique la règle sans vérification en dessous

B — MLM seul

Arbitrage MLM sur toutes les positions candidates, sans tenir compte de la confiance HTR

C — Combinée (recommandée)

Confiance HTR comme premier filtre, MLM seulement sous le seuil

Quelle stratégie l'emporte ?

- * Le CER mesuré sur vos 200 lignes de référence est la métrique d'ablation
- * La stratégie C (combinée) devrait dominer A et B
- * Mais ce n'est pas garanti : si le seuil est mal calibré, B peut être meilleur que C

C'est précisément ce que votre tableau d'ablation doit vérifier empiriquement, pas supposer.

PARTIE 6

Étape 4 — Fine-tuning mT5 LoRA sur les paires normalisées

Construire les paires d'entraînement

(transcription brute, forme normalisée)

- * Les paires proviennent de la sortie du pipeline de règles + abréviations sur le corpus
- * On ne garde que les lignes où le pipeline a effectivement changé quelque chose
- * Chaque paire porte aussi un poids (sample_weight = confidence HTR) et le document_type

Ce qui est nouveau ici : la construction des données. La mécanique LoRA a été expliquée au Chapitre 3.

Le pipeline complet, mot par mot

- 1 Normalisation Unicode
- 2 Alternances u/v et i/j
- 3 Règles graphémiques (terminaisons)
- 4 Résolution des abréviations
- 5 Lookup DMF (si disponible dans le cache)

Le résultat de `full_pipeline(text)` constitue la cible (target) pour le fine-tuning mT5 LoRA.

Trois métriques d'évaluation, complémentaires

Token Accuracy

% de mots correctement normalisés — métrique la plus simple à lire

CER normalisé

Erreurs au niveau caractère — la plus fine et la plus standard pour le domaine

BLEU-4

Chevauchement de n-grammes (jusqu'à 4) entre sortie et référence

Token Acc = (mots correctement normalisés) / (mots totaux)

BLEU-4 : la formule

$$BLEU-4 = BP \times \exp((1/4) \sum \log p_n)$$

- * p_n = précision modifiée des n-grammes (n = 1 à 4)
- * BP = pénalité de brièveté (Brevity Penalty), pour éviter qu'une sortie trop courte soit favorisée

Le tableau d'ablation du Jour 2

Sept configurations, du brut au pipeline hybride

Configuration	Params entraînés	Note
HTR brut (baseline)	0	Point de départ
Règles seules	0	Étape 1
Règles + abréviations	0	Étapes 1+2
Règles + MLM conf.	0	Étapes 1+2+3
mT5 LoRA r=8, Q+V	~300K	Étape 4
mT5 LoRA r=16, Q+V	~600K	Ablation r
Pipeline hybride	~300K	Règles → mT5

Le pipeline hybride : souvent le meilleur

- * Les règles traitent les cas déterministes et réduisent la fragmentation BPE (rappel Chapitre 1)
- * Le modèle neural ne traite que les cas ambigus restants
- * Hypothèse : la combinaison surpasse chaque approche isolée

Cette hypothèse n'est pas un acquis — c'est exactement ce que votre tableau d'ablation doit vérifier.

PARTIE 7

Traçabilité

CONVENTIONS_NLP.md et journal d'expériences

Pourquoi la traçabilité est un livrable

- * Les décisions de normalisation ont des conséquences scientifiques
- * Normaliser li roys en le roi efface une distinction morphosyntaxique (cas sujet vs cas régime)
- * Cette distinction peut être significative pour un historien ou un linguiste qui réutilisera vos données

Règle simple : toute décision non entièrement déterminée par le DMF doit figurer dans CONVENTIONS_NLP.md, avec sa justification.

Versionner le module de règles

Convention de versionnement sémantique

v1.0.0

Version initiale du module de règles

v1.0.1

Ajout d'une règle, sans modification de l'existant

v1.1.0

Modification d'une règle existante

Chaque modification change le corpus d'entraînement, change les métriques, et invalide les comparaisons antérieures.

Mettre à jour le journal d'expériences

experiments/journal.jsonl, introduit au Chapitre 3

```
log_experiment(  
    config={"step": 1, "method": "rules",  
           "rules_version": "v1.0.0",  
           "grapheme_rules": len(GRAPHEME_RULES)},  
    metrics={"cer_avant": cer_avant, "cer_apres": cer_apres,  
            "reduction": (cer_avant-cer_apres)/cer_avant},  
    split_hash=SPLIT_HASH,  
)
```

Le journal est mis à jour après chaque étape mesurée — pas seulement à la fin du TP.

Les livrables attendus en fin de séance

- * Le module de règles documenté (CONVENTIONS_NLP.md)
- * La table d'abréviations résolues
- * Le checkpoint du modèle de normalisation
- * Le tableau d'ablation complet
- * Le journal d'expériences mis à jour

Pour aller plus loin

Quelques repères bibliographiques

Linguistique & DMF

Marchello-Nizia (1997) ; Souvay & Pierrel (2009), LGeRM ; Dictionnaire du Moyen Français (ATILF)

Normalisation de textes historiques

Bollmann (2019), comparaison règles vs neural ; Clérice (2023), Pie Extended

Évaluation MLM

Salazar et al. (2020), Masked Language Model Scoring — fondement de l'arbitrage de l'Étape 3